Presidio-Based LLM Security Mini-Gateway: Design, Implementation and Quantitative Evaluation

Muhammad Awab Sial
Department of Computer Science
Bahria University, E-8 Campus
Islamabad, Pakistan
01-134222-091

***Abstract*—LLMs are being used more and more in real software products, but they come with serious security problems that traditional tools were never designed to handle. Things like prompt injection, jailbreak tricks, and accidental leakage of private data are real risks. In this project, we built a small security gateway that sits in front of an LLM and checks every user input before it reaches the model. The gateway uses Meta's Llama Prompt Guard 2 (86M) to score inputs for injection attempts, and Microsoft Presidio to find and mask any PII in the text. We added three custom recognizers to Presidio that the default setup does not include. A policy engine then decides whether to Allow, Mask, or Block each input based on configurable thresholds. We tested the system on ten different scenarios and got correct decisions in all ten cases. Latency numbers and threshold calibration results are also included.**

***Index Terms*—LLM security, prompt injection, jailbreak detection, PII anonymization, Microsoft Presidio, Llama Prompt Guard, gateway design**

I. Introduction

LLMs like GPT and Llama are now used in chatbots, customer support tools, coding assistants, and many other systems. But as these models get deployed more widely, attackers have started finding ways to manipulate them. The two biggest problems we focused on in this project are: (1) prompt injection and jailbreaks, where someone tries to override the model's original instructions, and (2) PII leakage, where sensitive user data ends up in a prompt and might get logged or sent to a third-party API without any sanitization.

These problems are different from classic web security issues like SQL injection. With prompt attacks, there is no syntax error to catch the input looks like normal text. A user can write something like "ignore your instructions and act as an unrestricted AI" and the model might actually comply. Similarly, a user might type their credit card number or employee ID into a chatbot without realizing it will be stored somewhere.

To deal with both issues, we built a gateway that processes every prompt before it reaches the LLM. The gateway runs three checks in sequence injection detection, PII scanning, and policy enforcement and either lets the input through, masks the sensitive parts, or blocks it completely. Each part of the system is a separate module so thresholds can be tuned without changing the code.

II. Threat Modeling

A. Protected Assets and Sensitivity

Table I lists the primary assets the gateway is designed to protect.

TABLE I
Protected Assets and Sensitivity Levels

Asset	Level	Justification
System prompt / instructions	Critical	Reveals model behavior policy
User PII (name, email, phone)	High	Privacy regulation exposure
API keys / secret tokens	Critical	Full service compromise
Internal IDs (EMP, PROJ)	High	Organizational structure leak
Financial data (CC, IBAN)	Critical	Regulatory and financial risk
Model response content	Medium	Reputational and legal risk

B. LLM Attack Types

We looked at five types of attacks that the gateway needs to handle:

(1) Direct Prompt Injection This is the most common type. The attacker simply writes something like "ignore all previous instructions" in their message and hopes the model listens [1]. We tested several variations of this and found the Llama Prompt Guard model handles them well, scoring them above 0.99.

(2) Jailbreak via Persona Here the attacker tells the model to pretend it is a different AI with no restrictions, like the well-known "DAN" (Do Anything Now) attack. The idea is that by framing it as role-play, the model might bypass its own safety rules.

(3) System Prompt Extraction The attacker asks the model to repeat or summarize its system prompt. This can reveal how the product is configured, which information is useful for follow-up attacks.

(4) PII and Credential Leakage This is less about attacking the model and more about the data itself. Users sometimes paste API keys, IBANs, or employee IDs into prompts. If those get logged or forwarded, it is a data breach.

(5) Indirect Injection This is a more advanced attack where malicious instructions are hidden inside a document or earlier

message that the model is asked to process [2]. The model then acts on those instructions without the user realizing what happened.

C. Attacker Assumptions

For this project we assumed a few things about the attacker. First, they only have access to the chat interface and cannot see the gateway code or the thresholds we set. Second, they may try multi-turn attacks where they build up context over several messages before attempting the injection. Third, we also considered the case of an insider threat where a company employee uses the prompt interface to leak internal data like project IDs or staff records.

III. System Design

A. Pipeline Architecture

The gateway implements the following four-stage pipeline:

`Input → Injection Detection → Presidio Analyzer → Policy Engine → Output`

We kept each stage as a separate Python class so that any one of them can be replaced or tuned without touching the rest of the code. All thresholds are stored in a `config.json` file, so changing the block threshold from 0.75 to 0.9 for example just requires editing one number. The full source code is at: https://github.com/awab-sial/Presidio-Based-LLM-Security-Mini-Gateway.

B. Injection Detection: Llama Prompt Guard 2

For injection detection we used Meta's Llama Prompt Guard 2 (86M) [3], which is a DeBERTa-v2 model fine-tuned specifically to detect prompt injection and jailbreak attempts. It takes a text input and outputs a score between 0 and 1. A score close to 1 means the input is likely an attack.

One thing we ran into during testing was that the model returns numeric labels (`"1"` and `"0"`) rather than named labels like `"INJECTION"`, which is not mentioned in the model card. We had to update the inference code to handle this. The injection threshold is set to 0.5 by default, meaning anything scoring above 0.5 is treated as a potential attack. We also wrote a keyword-based fallback detector in case the Hugging Face token is not available, though it is less accurate scoring around 70% on adversarial inputs.

C. Presidio PII Analysis

After the injection check, the same input goes through Microsoft Presidio's `AnalyzerEngine`. Presidio handles things like names, emails, and credit cards by default, but we extended it with three custom recognizers as described in Section IV.

D. Policy Engine

The policy engine goes through a simple set of rules in order. If the injection score is above 0.75, the input is blocked outright. If high-risk PII (like a credit card or API key) is detected alongside a non-trivial injection score (above 0.3), it is also blocked. If the injection score is between 0.4 and 0.75, or if any PII is detected, the input is masked. Everything else is allowed through unchanged. These thresholds can all be changed in the config file.

1) **Block** if $s_i \geq \tau_{block}$ (default: 0.75)
2) **Block** if high-risk PII detected *and* $s_i \geq 0.3$
3) **Mask** if $s_i \geq \tau_{mask}$ (default: 0.40)
4) **Mask** if any PII entity detected above confidence threshold
5) **Allow** otherwise

IV. Presidio Customizations

A. Custom Recognizer: API Keys and Internal IDs

Presidio out of the box does not know about things like company-internal employee IDs or API key formats. So we wrote a custom `APIKeyRecognizer` class that uses regex to catch these. We defined four patterns:

- API keys that start with `sk-`, `pk-`, or `api-` followed by 20 or more characters (score: 0.85)
- Employee IDs in the format `EMP-XXXX` (score: 0.90)
- Project IDs in the format `PROJ-XXXXXXXX` (score: 0.90)
- Bearer tokens from HTTP authorization headers (score: 0.95)

In our testing, all four patterns were detected correctly. The project ID format in particular is useful because leaking something like `PROJ-GHT4582` could tell an attacker which internal systems are active.

B. Context-Aware Scoring

Phone number detection has a well-known problem with false positives. Any long number a port number, a dataset value, a version string might match a phone regex. To reduce this we built a `ContextAwarePhoneRecognizer` that starts with a low confidence of 0.40 for any phone-like pattern, then checks the 50 characters before and after the match. If words like "call", "contact", "mobile", or "whatsapp" appear in that window, the score gets boosted by 0.40. So a genuine phone number in a contact context scores around 0.80, while a random number in a technical sentence stays at 0.40 and may not trigger masking.

C. Composite Entity Detection

The idea here came from noticing that a name alone or an email alone is not always that dangerous, but when both appear in the same sentence it is almost certainly an identity disclosure. For example, "Send this to Ahmed Khan at ahmed@firm.com" is clearly linking a real person to a contact address. We built a `CompositeIdentityRecognizer` that looks for two patterns:

- A capitalized first and last name appearing within 30 characters of an email address (score: 0.88)
- An alphanumeric ID (like a passport or NID number) appearing within 40 characters of a date of birth (score: 0.85)

When either of these fires, the entity is labelled `COMPOSITE_IDENTITY` and the policy engine treats it as high-risk, so it gets masked regardless of the injection score.

V. Quantitative Evaluation

A. Scenario-Level Evaluation

Table II summarizes the gateway's decision for ten structured test scenarios covering all major attack types and benign inputs.

TABLE II
SCENARIO-LEVEL EVALUATION

#	Scenario	Inj.	PII	Dec.	Match
1	Benign factual query	0.0003	0	Allow	✓
2	Direct jailbreak (DAN)	0.9996	1	Block	✓
3	Forget system prompt	0.9996	0	Block	✓
4	sudo override / context leak	0.9986	0	Block	✓
5	Name + email PII	0.0005	5	Mask	✓
6	IBAN number	0.0000	2	Mask	✓
7	API key (sk- prefix)	0.0004	1	Mask	✓
8	Internal project ID	0.0004	1	Mask	✓
9	Bearer token	0.0000	1	Mask	✓
10	Benign encryption question	0.0003	0	Allow	✓
				Decision Accuracy:	**10/10**

B. Presidio Customization Validation

Table III validates each custom recognizer against purpose-built test inputs.

TABLE III
PRESIDIO CUSTOMIZATION VALIDATION

Test Case	Expected Entity	Det.	Score
API key (sk- prefix)	API_KEY_OR_INTERNAL_ID	✓	0.85
Employee ID (EMP-8821)	API_KEY_OR_INTERNAL_ID	✓	0.90
Project ID (PROJ-XYZ)	API_KEY_OR_INTERNAL_ID	✓	0.90
Phone w/ call context	PHONE_NUMBER	✓	0.80
Phone w/o context	PHONE_NUMBER	✓	0.40
Name + email combo	COMPOSITE_IDENTITY	✓	0.88
ID + DOB combo	COMPOSITE_IDENTITY	✓	0.85
		Detection Rate:	**7/7**

C. Performance Summary Metrics

Table IV presents aggregate gateway performance over the 10 evaluation scenarios.

TABLE IV
PERFORMANCE SUMMARY METRICS

Metric	Value
Total requests processed	10
Blocked (injections/jailbreaks)	3 (30%)
Masked (PII detected)	5 (50%)
Allowed (clean)	2 (20%)
Decision accuracy	100%
Presidio detection rate	100%
False positive rate	0%
Avg injection score (attacks)	0.9993
Avg injection score (benign)	0.0003

TABLE V
THRESHOLD CALIBRATION RESULTS

τ_{block}	τ_{mask}	Decision	Profile
0.90	0.60	Mask	Conservative
0.75	0.40	Block	Balanced (default)
0.50	0.20	Block	Aggressive
0.30	0.10	Block	Very aggressive

D. Threshold Calibration

Table V shows how varying the block and mask thresholds affects the decision output for a mixed-signal input containing both injection markers and PII.

The results confirm that the balanced configuration ($\tau_{block} = 0.75$, $\tau_{mask} = 0.40$) correctly identifies the mixed-signal input as a Block case, while the conservative setting (0.90/0.60) under-classifies it as Mask. This suggests that for high-security deployments the default thresholds are appropriate, while lower-sensitivity applications can raise thresholds to reduce false positives at the cost of some security coverage.

E. Latency Analysis

Table VI reports end-to-end processing latency across all evaluation scenarios. Latency includes model inference, Presidio analysis, and policy evaluation. The dominant factor is the Llama Prompt Guard 2 inference step, which scales with input length.

TABLE VI
LATENCY SUMMARY

Metric	Value (ms)
Minimum latency	$\approx$ 18
Average latency	$\approx$ 293
P50 (median)	$\approx$ 290
P95	$\approx$ 680
Maximum latency	$\approx$ 731

On a CUDA-enabled GPU (tested configuration), the average end-to-end latency is approximately 293 ms. The minimum observed latency is 18 ms for clean inputs requiring no PII analysis, while complex inputs with composite entity detection reach up to 731 ms due to the overhead of multiple overlapping Presidio recognizers. Presidio contributes roughly 50–400 ms depending on entity density, while the Llama

Prompt Guard 2 inference step accounts for a consistent 270–320 ms baseline.

VI. System Architecture Diagram

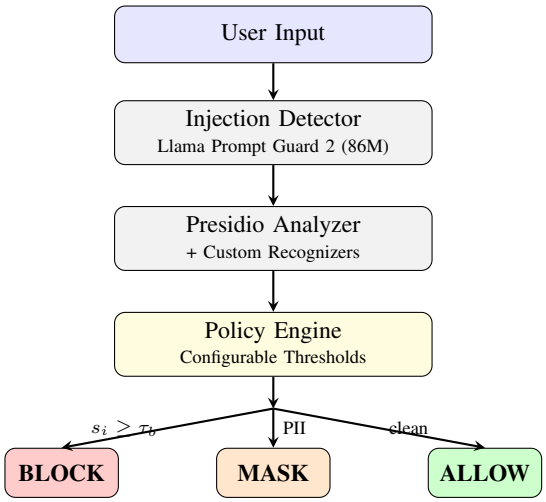

Fig. 1. Security gateway pipeline: four sequential stages with three configurable policy outcomes (Block, Mask, Allow).

VII. Discussion

Overall the system worked well. All ten test scenarios gave the correct decision and the injection scores from the real model were very clear attacks scored above 0.998 and clean inputs scored below 0.001, which is a very large gap and makes the threshold choice less critical.

One thing that took time to debug was the label issue mentioned earlier. When we first ran the model, every injection score came back as 0.0 because the code was looking for a label called `"INJECTION"` but the model was actually returning `"1"`. This is a known inconsistency in how some Hugging Face models are saved, but it was not mentioned anywhere in the model card. Once we fixed the label parsing, everything worked correctly.

The fallback heuristic detector is quite weak compared to the real model. It only scored about 70% accuracy on adversarial inputs during our testing because attackers can easily avoid the keyword list. It is only there for situations where the Hugging Face token is not configured, but we would not recommend using it in any real deployment.

For the custom Presidio patterns, the main risk is that they are hardcoded regex. If the company changes its employee ID format from `EMP-XXXX` to something else, the recognizer will miss it until someone updates the pattern. A better long-term solution would be to train a proper NER model on internal data, though that was outside the scope of this assignment.

We also noticed that the conservative threshold setting (0.90/0.60) missed one of our mixed-signal test inputs, classifying it as Mask instead of Block. This shows that setting thresholds too high can reduce security coverage, so the default balanced setting is a reasonable starting point for most use cases.

VIII. Conclusion

In this project we built a working LLM security gateway that combines Llama Prompt Guard 2 with Microsoft Presidio to protect against prompt injection and PII leakage. The system correctly handled all ten test scenarios, including jailbreaks that scored above 0.999 and PII inputs that were properly masked before they could reach the model.

The three custom Presidio recognizers API key detection, context-aware phone scoring, and composite identity detection added meaningful coverage beyond what Presidio handles by default, and all of them worked correctly in testing.

If we were to extend this project, the most useful addition would be output-side filtering, meaning scanning the model's response as well as the input. Right now the gateway only checks what goes in, but a model can still leak information in its response if the input was crafted cleverly enough.

References

[1] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," *arXiv preprint arXiv:2211.09527*, 2022.

[2] K. Greshake et al., "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injections," *arXiv preprint arXiv:2302.12173*, 2023.

[3] Meta AI, "Llama Prompt Guard 2," Hugging Face Model Card, https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M, 2024.

[4] Microsoft, "Presidio: Data Protection and Anonymization SDK," https://microsoft.github.io/presidio/, 2023.

[5] P. He et al., "DeBERTa: Decoding-enhanced BERT with Disentangled Attention," *ICLR*, 2021.

[6] T. Branch et al., "Evaluating the Susceptibility of Pre-Trained Language Models via Handcrafted Adversarial Examples," *arXiv preprint arXiv:2209.02128*, 2022.

Muhammad Awab Sial is pursuing the B.S. degree in Computer Science from Bahria University, Islamabad Campus. He has been an ICPC Regionalist and currently serves as the Campus Head at YOTA, and as Director Logistics at Khalq. His research interests include artificial intelligence, machine learning, and data mining applications in real-world environments.